

2025. 2 QUESTÃO 01

```
// Cliente
class Client {
    int clientCPF;
    String name;
    public Client(int clientCPF, String name) {
        this.clientCPF = clientCPF;
        this.name = name;
    }
}

// Pedido
class Order {
    int clientCPF;
    float value;
    public Order(int clientCPF, float value) {
        this.clientCPF = clientCPF;
        this.value = value;
    }
}

public class Sistema {
    private Client[] clientes;
    private Order[] pedidos;

    public Sistema(ArrayList clientes, ArrayList pedidos) {
        this.clientes = (Client[]) clientes.selectAll();
        this.pedidos = (Order[]) pedidos.selectAll();
    }

    // Só precisavam implementar este método na prova.
    // Coloquei toda a estrutura para facilitar a compreensão.
    public float masterDetail(String name) {
        // procura CPF do cliente com nome "name"
        Integer cpfBusca = null;
        for (Client c: clientes)
            if (c.name.equals(name)) {
                cpfBusca = c.clientCPF;
                break;
            }

        float soma = 0.0;
        if (cpfBusca != null) {
            for (Order o : pedidos)
                if (o.clientCPF == cpfBusca)
                    soma += o.value;
        }
        return soma;
    }

    public static void main(String[] args) {
        ArrayList clientes = new ArrayList(10);
        ArrayList pedidos = new ArrayList(10);

        clientes.append(new Client(123, "J"));
        clientes.append(new Client(156, "M"));
        clientes.append(new Client(246, "A"));

        pedidos.append(new Order(123, 100.0));
        pedidos.append(new Order(156, 1000.5));
        pedidos.append(new Order(156, 200.0));
        pedidos.append(new Order(123, 300.3));
        pedidos.append(new Order(123, 10.2));

        Sistema sys = new Sistema(clientes, pedidos);
        System.out.println("Total para J: " + sys.masterDetail("J")); // espera 410.5
        System.out.println("Total para M: " + sys.masterDetail("M")); // espera 1200.5
        System.out.println("Total para A: " + sys.masterDetail("A")); // espera 0.0
    }
}
```

QUESTÃO 2

```
public class Questao2 {
    String[] alunosA;
    String[] alunosB;
    String[] alunosC;

    public Questao2(ArrayList A, ArrayList B, ArrayList C) {
        alunosA = (String[]) A.selectAll();
        alunosB = (String[]) B.selectAll();
        alunosC = (String[]) C.selectAll();
    }

    // Só precisavam implementar estes métodos na prova. // Coloquei toda a estrutura para facilitar a compreensão.
    private boolean contains(String nome, String[] alunos) {
        for(String a: alunos)
            if (a.equals(nome))
                return true;

        return false;
    }

    /** a) alunos que fazem esportiva e cultural e não extensão */
    public String[] culturaEEsporteSemExtensao() {

        ArrayList result = new ArrayList();
        for (String aluno : alunosB) {
            if (contains(aluno, alunosA) && !contains(aluno, alunosC)) {
                result.append(aluno);
            }
        }
        return (String[]) result.selectAll();
    }

    /** b) alunos que fazem somente extensão */
    public String[] somenteExtensao() {

        ArrayList result = new ArrayList();
        for (String aluno : alunosC) {
            if (!contains(aluno, alunosA) && !contains(aluno, alunosB)) {
                result.append(aluno);
            }
        }
        return (String[]) result.selectAll();
    }

    /** c) alunos que fazem cultural ou extensão, mas não fazem esportiva */
    public String[] culturaOuExtensaoSemEsporte() {

        ArrayList result = new ArrayList();
        // incluir os de A que não estão em B
        for (String aluno : alunosA) {
            if (!contains(aluno, alunosB) && !contains(aluno, result)) {
                result.append(aluno);
            }
        }
        // incluir os de C que não estão em B, e que ainda não foram incluídos
        for (String aluno : alunosC) {
            if (!contains(aluno, alunosB) && !contains(aluno, result)) {
                result.append(aluno);
            }
        }
        return (String[]) result.selectAll();
    }

    public static void main(String[] args) {
        ArrayList culturaA = new ArrayList();
```

```

culturaA.append("Alice");
culturaA.append("Bob");
culturaA.append("Carol");
culturaA.append("David");

```

```

ArrayList esporteB = new ArrayList();
esporteB.append("Bob");
esporteB.append("David");
esporteB.append("Eve");
esporteB.append("Frank");

```

```

ArrayList extensaoC = new ArrayList();
extensaoC.append("Carol");
extensaoC.append("David");
extensaoC.append("Frank");
extensaoC.append("Gael");

```

```

Questao2 q2 = new Questao2(culturaA, esporteB, extensaoC);
System.out.println("a) Esporte + Cultura, sem Extensão: " + q2.culturaEEsporteSemExtensao()); // Bob
System.out.println("b) Somente Extensão: " + q2.somenteExtensao()); // Gael
System.out.println("c) Cultura ou Extensão, sem Esporte: " + q2.culturaOuExtensaoSemEsporte()); // Alice, Caro, Gael
}
}

```

QUESTÃO 3

```

public class Questao3 {

```

```

    class Patient {
        String name;
        int age;
        int risk;

```

```

        public Patient(String name, int age, int risk) {
            this.name = name;
            this.age = age;
            this.risk = risk;
            setPriority(risk);
        }

```

```

        // Só precisavam implementar estes códigos na prova. // Coloquei toda a estrutura para facilitar a compreensão.
        long priority;
        static long counter = 0;

```

```

        public void setPriority(int risk) {
            this.priority = priorityAdjustment(risk);
        }

```

```

        public long priorityAdjustment(int risk) {
            int incremento = 999_999 - counter;
            counter++;
            return risk * 1_000_000 + incremento;
        }

```

```

    }

```

```

    public static void main(String[] args) {
        BinaryHeap filaPacientes = new BinaryHeap();
        filaPacientes.enqueue(new Patient("João", 30, 1));
        filaPacientes.enqueue(new Patient("Pedro", 40, 2));
        filaPacientes.enqueue(new Patient("Lucas", 28, 3));
        filaPacientes.enqueue(new Patient("Maria", 25, 2));
        filaPacientes.enqueue(new Patient("Ana", 35, 3));

        System.out.println("Atendendo pacientes por ordem de prioridade:");
        while (!filaPacientes.isEmpty()) {
            Patient p = (Patient) filaPacientes.dequeue();
            System.out.println("Paciente: " + p.name);
        }
    }
}

```